CreatorFlow AI

Software Design Specification

Document Type: Software Design Specification (SDS)
Project: CreatorFlow AI — AI-Powered Content Optimization Platform
Version: 1.0
Prepared by: Michael Berry
Date: June 2026
Status: Draft — Submitted for Review
Course: Software Engineering II

© 2026 CreatorFlow AI. All rights reserved.

Table of Contents

1. Introduction 3
- 1.1 Project Overview 3
- 1.2 Purpose and Objectives of This Document 3
- 1.3 Importance of Design Before Development 3
- 1.4 Scope 3

2. System Architecture 4
- 2.1 Architecture Overview 4
- 2.2 High-Level Architecture Diagram 4
- 2.3 Tier Descriptions 4
- 2.4 Data Flow 5

3. User Interface Design 6
- 3.1 Design Principles 6
- 3.2 Color Palette and Typography 6
- 3.3 Global Layout 6
- 3.4 Screen Wireframes 7

4. Database Design 9
- 4.1 Database Technology 9
- 4.2 Entity-Relationship Overview 9
- 4.3 Entity Schemas 9
- 4.4 Firestore Security Rules 11
- 4.5 Data Access Patterns 11

5. Component Design 12
- 5.1 Component Overview 12
- 5.2 Frontend Component Map 12
- 5.3 Backend Component Map 13
- 5.4 Key Sequence Diagrams 13

6. API Design 15
- 6.1 API Overview 15
- 6.2 Endpoint Reference 15
- 6.3 Request and Response Contracts 16
- 6.4 Rate Limiting and Throttling 17

7. Security Design 18
- 7.1 Security Architecture Overview 18
- 7.2 Authentication 18
- 7.3 Authorization 18
- 7.4 Credential and Secret Management 19
- 7.5 Transport Security 19
- 7.6 Input Validation and Injection Prevention 20
- 7.7 Security Checklist 20

8. Design Review and Feedback 21
- 8.1 Design Review Process 21
- 8.2 Review Session Notes 21
- 8.3 Revisions Made 22

© 2026 CreatorFlow AI. All rights reserved.

8.4 Post-Review Design Confidence22

Document Revision History23

© 2026 CreatorFlow AI. All rights reserved.

1. Introduction

1.1 Project Overview

CreatorFlow AI is a full-stack, web-based AI productivity platform built for independent digital content creators operating across YouTube, TikTok, Instagram, and adjacent creator ecosystems. The platform integrates the OpenAI API (GPT-3.5) to automate high-value but time-intensive content optimization tasks — including title generation, description writing, hashtag research, and SEO keyword recommendations — enabling creators to focus on production quality rather than metadata overhead.

The application is built on a modern JavaScript stack: a React 18 frontend deployed on Vercel, a Node.js + Express backend hosted on Render, Firebase for authentication and cloud data persistence, and the OpenAI API for AI-powered generation. Development follows an Agile Scrum methodology structured into six two-week sprints, targeting a polished Minimum Viable Product (MVP) as the primary deliverable.

1.2 Purpose and Objectives of This Document

This Software Design Specification (SDS) translates the requirements defined in the SRS into a concrete, actionable design blueprint for CreatorFlow AI. It bridges the gap between what the system must do (requirements) and how the system will be built (architecture, components, data models, and interfaces).

The specific objectives of this document are:

- Define the system architecture and how all components interact across client, server, and data tiers
- Provide detailed UI wireframe descriptions for all major application screens
- Specify the Firebase Firestore database schema, entity relationships, and data access patterns
- Describe each major system component, its responsibilities, and its collaboration with other components
- Document the REST API design, including all endpoints, request/response contracts, and authentication flows
- Define the security architecture, including authentication, authorization, encryption, and credential management
- Record the design review process, feedback received, and revisions made

1.3 Importance of Design Before Development

Why Design First?

A well-defined design specification serves as the authoritative blueprint for every implementation decision made during development. Without it, individual sprints risk producing components that are technically functional in isolation but architecturally inconsistent with each other — leading to integration failures, rework, and scope creep. For a solo developer working across a 12-week sprint cycle with a fixed academic deadline, design-first discipline is especially critical: there is no team to catch architectural drift, no dedicated QA phase to surface integration gaps, and no buffer for major rework in late sprints. This SDS locks in the technical decisions that would otherwise be rediscovered — inconsistently — across multiple sprints.

© 2026 CreatorFlow AI. All rights reserved.

1.4 Scope

Scope Category	Items Covered
In Scope	Authentication system, AI Title Generator, AI Description Generator, AI Hashtag Generator, AI SEO Optimizer, Project Save System, REST API, Firebase Firestore schema, security design
Out of Scope (v1.0)	Payment processing, native mobile apps, real-time collaboration, social media publishing, custom AI model training, advanced video analysis

© 2026 CreatorFlow AI. All rights reserved.

2. System Architecture

2.1 Architecture Overview

CreatorFlow AI follows a three-tier client-server architecture with a clear separation of concerns between presentation, business logic, and data layers. The architecture is intentionally designed for independent deployability — the React frontend, Node.js backend, and Firebase data tier can each be scaled, updated, or replaced without cascading changes to the other layers.

2.2 High-Level Architecture Diagram

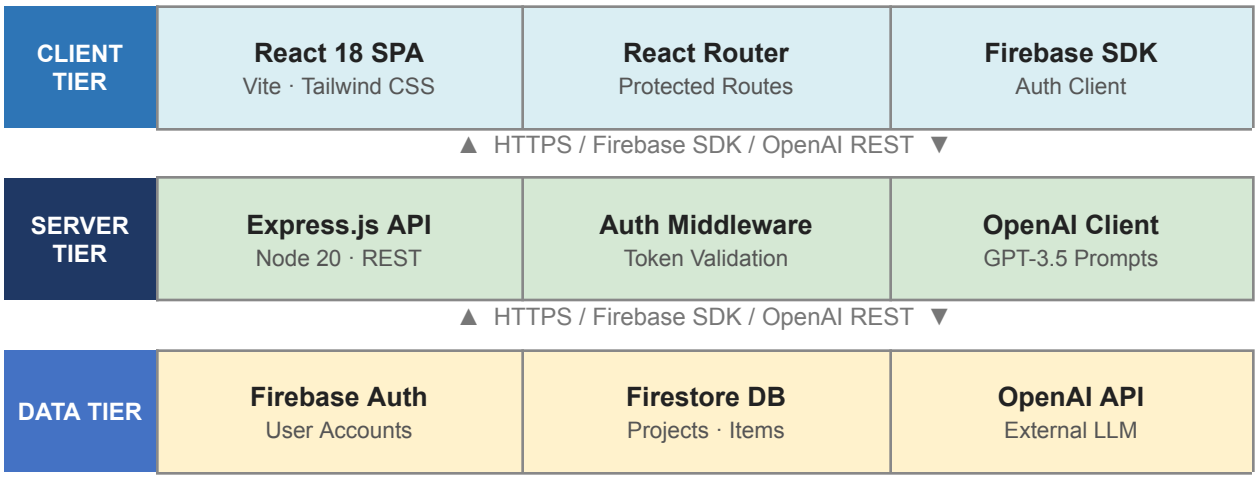


Figure 1 — CreatorFlow AI Three-Tier System Architecture

2.3 Tier Descriptions

2.3.1 Client Tier — React Frontend

The client tier is a React 18 single-page application (SPA) built with Vite for tooling and Tailwind CSS for styling. It is responsible for all user interaction, form rendering, state management, and result display. The frontend communicates exclusively with the Express backend via authenticated HTTPS REST calls — it never calls the OpenAI API directly, ensuring API keys remain server-side at all times.

Framework	React 18 (SPA)
Build Tool	Vite
Styling	Tailwind CSS
Routing	React Router v6 (protected routes)
Auth Client	Firebase SDK v10 (client-side token acquisition)
Hosting	Vercel (auto-deploy from GitHub main branch)

2.3.2 Server Tier — Node.js + Express Backend

The server tier is a Node.js 20 application using the Express.js framework. It acts as the secure intermediary between the client and all external services. It validates Firebase ID tokens on every authenticated request, constructs and sanitizes OpenAI prompts, manages error handling, and returns structured generation results to the frontend.

© 2026 CreatorFlow AI. All rights reserved.

Runtime	Node.js 20
Framework	Express.js
Auth Middleware	Firebase Admin SDK (server-side token verification)
AI Client	OpenAI Node SDK (GPT-3.5-turbo)
CORS	cors npm package — configured for Vercel frontend origin only
Hosting	Render (managed Node.js — auto-deploy from GitHub)

2.3.3 Data Tier — Firebase Platform

The data tier consists of two Firebase services: Firebase Auth for identity management and Firebase Firestore for cloud-hosted NoSQL data persistence. Firestore security rules enforce that users can only read and write documents they own, providing row-level access control without custom authorization middleware.

Auth Service	Firebase Authentication (email/password + OAuth)
Database	Firebase Firestore (serverless NoSQL — document model)
Security Rules	Firestore Security Rules (owner-scoped read/write)
SDK	Firebase Admin SDK (server) + Firebase SDK v10 (client)

2.4 Data Flow

The following table documents the end-to-end data flow for an AI content generation request — the core workflow of the CreatorFlow AI platform:

#	Step	Description
1	**User Input**	Creator submits topic, platform, and tone via React form
2	**Auth Request**	Frontend attaches Firebase ID token to POST /api/generate
3	**Token Validate**	Express middleware verifies token with Firebase Admin SDK
4	**Prompt Build**	Backend constructs sanitized prompt from validated inputs
5	**OpenAI Call**	Backend POSTs to /v1/chat/completions with GPT-3.5 model
6	**Response Parse**	Backend parses JSON response and extracts generated content
7	**Return to Client**	Express returns structured result to React frontend
8	**Display + Save**	UI renders results; user optionally saves to Firestore

Figure 2 — AI Generation Request Data Flow

© 2026 CreatorFlow AI. All rights reserved.

3. User Interface Design

3.1 Design Principles

The CreatorFlow AI interface is designed around four core UX principles that govern every screen, interaction, and layout decision:

Principle	Definition	Application in CreatorFlow AI
Consistency	Identical layout patterns across all tool pages	All generation tools share the same three-section layout: input → generate → results
Feedback	The UI always communicates system state to the user	Loading spinners during API calls; success/error toasts on save actions
Hierarchy	Visual weight guides user attention through tasks	Navy headers → Blue subheadings → Body text creates natural reading flow
Accessibility	Usable across ability levels and device sizes	WCAG 2.1 AA contrast ratios; keyboard navigation; responsive 375px–1920px

3.2 Color Palette and Typography

Element	Value	Usage
Primary Navy	#1E3A5F / #1F3864	Page headers, sidebar background, primary buttons
Accent Blue	#2E75B6	Section headings, active nav items, interactive elements
Light Blue	#4A9FD4	Hover states, highlights, tag chips
Table Fill	#D5E8F0	Alternating table rows, info boxes
Body Text	#404040	All standard paragraph text
Font Family	Arial / System Sans-serif	Applied universally — no web font dependency
Base Size	16px (12pt)	Body text; headings scale proportionally

3.3 Global Layout

All authenticated screens share a persistent two-panel layout: a fixed-width left sidebar (240px) for navigation and a fluid main content area. On mobile viewports (< 768px), the sidebar collapses behind a hamburger icon and the main content expands to full width.

Sidebar	Fixed 240px width · Navy (#1F3864) background · user avatar + name · nav links with active highlight
Main Content	Fluid width · white background · 24px padding · max-width 1200px centered
Header Bar	Sticky top · 64px height · page title left · user menu right
Mobile Breakpoint	< 768px: sidebar hides behind hamburger · content fills viewport · bottom nav tab bar

© 2026 CreatorFlow AI. All rights reserved.

3.4 Screen Wireframes

3.4.1 Login / Registration Screen

🖥️ Login · Registration
[Logo + Tagline] CreatorFlow AI logo centered at top — 'AI-Powered Content Optimization for Creators'
[Auth Form] Email field + Password field + Submit button (full-width) + Toggle link: 'No account? Register'
[Registration Form] Email + Password + Confirm Password + Register button — validation inline below each field
[Error State] Red inline text below affected field — e.g., 'Password must be at least 8 characters'
[Success State] Redirect to Dashboard on successful authentication — no manual confirmation step required
© 2026 CreatorFlow AI | Page [N]

Figure 3 — Login / Registration Screen Wireframe

3.4.2 Creator Dashboard

🖥️ Creator Dashboard
[Left Sidebar] Nav links: Dashboard · Generate Titles · Generate Descriptions · Generate Hashtags · SEO Suggestions · Saved Projects · [Logout]
[Welcome Banner] Greeting header — 'Welcome back, [Name]' — with today's date
[Quick Actions Row] Four CTA cards: Generate Titles · Generate Descriptions · Generate Hashtags · SEO Suggestions — each with icon and link
[Recent Projects] Grid of last 5 saved project cards — project name, type badge, timestamp, delete icon
[Generation Stats] Count of total generations this session — informational only
© 2026 CreatorFlow AI | Page [N]

Figure 4 — Creator Dashboard Wireframe

© 2026 CreatorFlow AI. All rights reserved.

3.4.3 AI Generation Tool Pages (Shared Pattern)

All four AI generation tool pages (Title Generator, Description Generator, Hashtag Generator, SEO Optimizer) share the identical three-section layout described below. Only the input fields and result format differ per tool.

AI Generation Tool (Shared Layout)

[Section 1 — Input Form]
Labeled fields specific to the tool (topic, platform dropdown, tone dropdown where applicable) · All fields include placeholder text and inline validation

[Generate Button]
Full-width Navy button — 'Generate [Content Type]' — disabled until required fields are filled · Shows spinner + 'Generating…' during API call

[Section 2 — Loading State]
Skeleton card placeholders animate while awaiting OpenAI API response — prevents layout shift on result load

[Section 3 — Results Area]
Results rendered as individual cards — each card contains: generated content · one-click Copy button · Save to Project button · character/word count badge

[Empty State]
Illustrated empty state with prompt text — 'Fill in the form above and click Generate to get started'

[Error State]
Red error banner below generate button — 'Something went wrong. Please try again.' with retry button

© 2026 CreatorFlow AI | Page [N]

Figure 5 — AI Generation Tool Shared Layout Wireframe

3.4.4 Saved Projects Screen

Saved Projects

[Search + Filter Bar]
Text search input · Filter dropdown by content type (Title / Description / Hashtags / SEO) · Sort by: Newest / Oldest

[Project Cards Grid]
Responsive grid — each card shows: content type badge (color-coded) · first 120 chars of content · platform tag · timestamp · Delete icon (with confirmation)

[Empty State]
Centered illustration + message: 'No saved content yet. Generate something and save it here.'

[Delete Confirmation]
Inline confirmation — 'Are you sure? This cannot be undone.' with Confirm and Cancel buttons

© 2026 CreatorFlow AI | Page [N]

Figure 6 — Saved Projects Screen Wireframe

© 2026 CreatorFlow AI. All rights reserved.

4. Database Design

4.1 Database Technology

CreatorFlow AI uses Firebase Firestore — a serverless, cloud-hosted NoSQL document database — as its sole persistence layer. Firestore organizes data into collections of documents, where each document is a JSON-like object that can contain nested subcollections. This model aligns naturally with the application's data structure, where each user owns a set of projects and each project contains a set of generated items.

Database Type	NoSQL Document Database (Firebase Firestore)
Data Model	Collections → Documents → Subcollections
Access Control	Firestore Security Rules (owner-scoped: userId match)
SDK	Firebase Admin SDK (server) + Firebase Client SDK v10 (read queries)
Environment	Firebase Spark (free tier) — sufficient for development and MVP

4.2 Entity-Relationship Overview

The database contains three primary entities: User (managed by Firebase Auth with a corresponding Firestore profile document), Project (a named container for generated content), and GeneratedItem (an individual AI output saved by the user). The relationships are:

- One User → zero or many Projects (1:N)
- One Project → zero or many GeneratedItems (1:N)
- GeneratedItem belongs to exactly one Project (N:1)

4.3 Entity Schemas

4.3.1 Users Collection (/users/{userId})

⬛ users /users/{userId}			
Field	**Type**	**Key**	**Notes**
userId	string	**PK**	Firebase Auth UID — document ID
email	string		User's email address from Firebase Auth
displayName	string		User's display name — set on registration
createdAt	timestamp		Firestore server timestamp on document creation
updatedAt	timestamp		Updated on any profile change

© 2026 CreatorFlow AI. All rights reserved.

4.3.2 Projects Collection (/users/{userId}/projects/{projectId})

⬛ projects /users/{userId}/projects/{projectId}			
Field	**Type**	**Key**	**Notes**
projectId	string	**PK**	Firestore auto-generated document ID
userId	string	**FK**	Reference to parent /users/{userId} — used in security rules
name	string		User-defined project name — max 100 chars
createdAt	timestamp		Firestore server timestamp on creation
updatedAt	timestamp		Updated on any save action within the project
itemCount	number		Denormalized count of saved items — updated on write

4.3.3 GeneratedItems Subcollection (/users/{userId}/projects/{projectId}/items/{itemId})

⬛ items /projects/{projectId}/items/{itemId}			
Field	**Type**	**Key**	**Notes**
itemId	string	**PK**	Firestore auto-generated document ID
projectId	string	**FK**	Reference to parent project document
type	string		Enum: 'title' | 'description' | 'hashtags' | 'seo'
content	string		The full AI-generated text content
platform	string		Target platform: 'YouTube' | 'TikTok' | 'Instagram'
topic	string		The user-provided topic input that triggered generation
tone	string		Tone parameter (titles only): 'Professional' | 'Casual' | 'Hype'
createdAt	timestamp		Firestore server timestamp on item save

© 2026 CreatorFlow AI. All rights reserved.

4.4 Firestore Security Rules

The following security rules enforce owner-scoped access — a user may only read or write documents in their own subcollection. These rules are deployed to Firebase and validated using the Firebase Rules Emulator before production deployment.

Firestore Security Rule Pattern

match /users/{userId}/{document=**} { allow read, write: if request.auth != null && request.auth.uid == userId; } This single recursive rule covers /users, /users/projects, and /users/projects/items in one declaration, ensuring no document in the user's subtree is accessible to any other authenticated user.

4.5 Data Access Patterns

Operation	Firestore Query	Called From
Load user dashboard	collection('users/{uid}/projects').orderBy('updatedAt', 'desc').limit(5)	Dashboard component on mount
Save generated item	collection('users/{uid}/projects/{pid}/items').add({...itemData})	Save button onClick handler
List all saved items	collection('users/{uid}/projects/{pid}/items').orderBy('createdAt', 'desc')	Saved Projects page
Delete saved item	doc('users/{uid}/projects/{pid}/items/{itemId}').delete()	Delete confirmation handler
Create new project	collection('users/{uid}/projects').add({name, createdAt})	Save dialog on first save

© 2026 CreatorFlow AI. All rights reserved.

5. Component Design

5.1 Component Overview

CreatorFlow AI is structured as a set of loosely coupled components divided across the frontend (React) and backend (Express). Each component has a single, well-defined responsibility and communicates with other components through defined interfaces — HTTP REST for frontend-to-backend, Firebase SDK for auth/data operations, and the OpenAI SDK for AI generation.

5.2 Frontend Component Map

Component	Type	Responsibility
App.jsx	Root	React Router setup; global auth state provider; route guard logic
AuthContext.jsx	Context	Firebase Auth state listener; exposes user object and auth methods to all children
ProtectedRoute.jsx	HOC	Redirects unauthenticated users to /login; wraps all dashboard routes
Sidebar.jsx	Layout	Fixed navigation panel; highlights active route; logout button
Dashboard.jsx	Page	Renders recent projects summary and quick action cards
TitleGenerator.jsx	Page	Input form + generate handler + results display for title tool
DescriptionGenerator.jsx	Page	Input form + generate handler + results display for description tool
HashtagGenerator.jsx	Page	Input form + generate handler + results display for hashtag tool
SeoOptimizer.jsx	Page	Input form + generate handler + results display for SEO tool
SavedProjects.jsx	Page	Lists all saved items; search/filter controls; delete handler
ResultCard.jsx	Component	Reusable card: displays one generated result with copy + save actions
useGenerate.js	Hook	Shared custom hook: manages API call state, loading, error for all tool pages
api.js	Service	Axios client: attaches Firebase ID token to all outbound API requests

5.3 Backend Component Map

Module	Type	Responsibility
server.js	Entry Point	Express app initialization; middleware registration; port binding
authMiddleware.js	Middleware	Verifies Firebase ID token on every protected route; attaches uid to req

© 2026 CreatorFlow AI. All rights reserved.

Module	Type	Responsibility
generateRoutes.js	Router	Mounts all /api/generate/* route handlers
generateController.js	Controller	Validates input; builds OpenAI prompt; calls openaiService; returns result
openaiService.js	Service	Encapsulates OpenAI SDK calls; handles retries and error mapping
promptBuilder.js	Utility	Constructs tool-specific prompts from sanitized user inputs
errorHandler.js	Middleware	Global Express error handler; maps errors to HTTP status codes
firebaseAdmin.js	Config	Initializes Firebase Admin SDK from environment variables

5.4 Key Sequence Diagrams

5.4.1 Authentication Flow

Step	From	To	Action / Message
1	User	LoginPage.jsx	Submits email + password form
2	LoginPage.jsx	Firebase Auth SDK	signInWithEmailAndPassword(email, password)
3	Firebase Auth	LoginPage.jsx	Returns UserCredential + ID token on success
4	LoginPage.jsx	AuthContext.jsx	Sets authenticated user in context state
5	AuthContext.jsx	React Router	Redirects to /dashboard via ProtectedRoute
6	Dashboard.jsx	Firestore SDK	Loads user's recent projects on mount

Figure — User Login Sequence Diagram

5.4.2 AI Generation Flow

Step	From	To	Action / Message
1	User	GeneratorPage.jsx	Fills form and clicks Generate
2	GeneratorPage.jsx	useGenerate.js	Calls generate(inputs) — sets loading: true
3	useGenerate.js	Firebase Auth SDK	getIdToken() — retrieves current auth token
4	useGenerate.js	api.js (Axios)	POST /api/generate/{type} with token + inputs
5	api.js	Express Server	HTTP request with Authorization: Bearer {token}
6	authMiddleware.js	Firebase Admin SDK	verifyIdToken(token) — validates and decodes

© 2026 CreatorFlow AI. All rights reserved.

Step	From	To	Action / Message
7	generateController.js	promptBuilder.js	buildPrompt(type, inputs) — sanitized prompt
8	generateController.js	openaiService.js	callOpenAI(prompt) — GPT-3.5 API call
9	openaiService.js	OpenAI API	POST /v1/chat/completions — awaits response
10	openaiService.js	generateController.js	Returns parsed content array
11	Express Server	useGenerate.js	HTTP 200 with { results: [...] }
12	GeneratorPage.jsx	ResultCard.jsx	Renders result cards — loading: false

Figure — AI Content Generation Sequence Diagram

© 2026 CreatorFlow AI. All rights reserved.

6. API Design

6.1 API Overview

CreatorFlow AI exposes a RESTful HTTP API served by the Node.js + Express backend. All endpoints that return user-specific data or trigger AI generation require authentication via a Firebase ID token passed in the Authorization request header. The API follows standard REST conventions: nouns for resource names, HTTP verbs for operations, and JSON for all request and response bodies.

Base URL (Production)	https://creatorflow-api.onrender.com/api
Base URL (Development)	http://localhost:5000/api
Protocol	HTTPS (production) / HTTP (local dev)
Auth Scheme	Bearer token — Authorization: Bearer {Firebase ID Token}
Content Type	application/json (all requests and responses)
API Versioning	Path-based versioning reserved for future use (/v1/) — v1.0 omits prefix

6.2 Endpoint Reference

6.2.1 Health Check

Method	Endpoint	Auth	Description
GET	/health	None	Returns API status — used by Render health monitor

6.2.2 AI Generation Endpoints

Method	Endpoint	Auth	Description
POST	/api/generate/titles	Bearer Token	Generate 3–5 optimized titles from topic, platform, and tone inputs
POST	/api/generate/descriptions	Bearer Token	Generate a platform-specific SEO description from topic and platform
POST	/api/generate/hashtags	Bearer Token	Generate 10–20 relevant hashtags from topic or description input
POST	/api/generate/seo	Bearer Token	Generate SEO keywords and optimization recommendations from topic

6.3 Request and Response Contracts

6.3.1 POST /api/generate/titles

Field	Location	Type	Required	Description
Authorization	Header	string	Yes	Bearer {Firebase ID Token}

© 2026 CreatorFlow AI. All rights reserved.

Field	Location	Type	Required	Description
topic	Body	string	Yes	Video or content topic — max 500 chars
platform	Body	string	Yes	Enum: 'YouTube' | 'TikTok' | 'Instagram'
tone	Body	string	Yes	Enum: 'Professional' | 'Casual' | 'Hype'

Success Response — HTTP 200

{ "results": ["10 JavaScript Tricks Senior Developers Actually Use", "You've Been Writing JavaScript Wrong (Here's Why)", "JavaScript Tips That Will Change How You Code Forever"], "generationType": "titles", "platform": "YouTube" }

6.3.2 POST /api/generate/hashtags

Field	Location	Type	Required	Description
Authorization	Header	string	Yes	Bearer {Firebase ID Token}
topic	Body	string	Yes	Content topic or description — max 500 chars
platform	Body	string	Yes	Enum: 'YouTube' | 'TikTok' | 'Instagram'

Success Response — HTTP 200

{ "results": ["#javascript", "#webdevelopment", "#coding", "#programming", "#techlife", "#developer", "#learntocode", "#js"], "generationType": "hashtags", "count": 8 }

6.3.3 Error Response Format

All endpoints return a consistent error response structure regardless of error type. HTTP status codes follow REST conventions: 400 for validation errors, 401 for authentication failures, 429 for rate limit errors, and 500 for server-side failures.

Error Response Format

{ "error": { "code": "INVALID_INPUT", "message": "The 'platform' field must be one of: YouTube, TikTok, Instagram.", "status": 400 } } Error Codes: UNAUTHORIZED, INVALID_INPUT, RATE_LIMITED, OPENAI_ERROR, INTERNAL_ERROR

6.4 Rate Limiting and Throttling

Limit Type	Value	Enforcement
Per-user generation rate	20 requests / hour	Express rate-limit middleware keyed on Firebase UID

© 2026 CreatorFlow AI. All rights reserved.

Limit Type	Value	Enforcement
OpenAI API budget cap	$20 development cap	Usage alert set at $15 in OpenAI dashboard — hard stop at $20
Request body size	10KB maximum	Express body-parser limit
Response timeout	30 seconds	Express route timeout — returns 504 if exceeded

© 2026 CreatorFlow AI. All rights reserved.

7. Security Design

7.1 Security Architecture Overview

Security in CreatorFlow AI is designed across four layers: authentication (verifying who the user is), authorization (enforcing what they can access), credential management (protecting API keys and secrets), and transport security (encrypting all data in transit). The principle of least privilege is applied throughout — each layer grants only the minimum permissions required for its function.

7.2 Authentication

User identity is managed exclusively by Firebase Authentication, a production-grade identity platform that handles credential storage, password hashing, session management, and OAuth flows. The application never stores, transmits, or handles raw user passwords at any point.

Auth Flow Step	Implementation Detail
Registration	Firebase Auth createUserWithEmailAndPassword() — Firebase hashes and stores credentials
Login	Firebase Auth signInWithEmailAndPassword() — returns a signed ID token (JWT) valid for 1 hour
Session Persistence	Firebase Auth persists session in IndexedDB — survives page refresh
Token Refresh	Firebase SDK auto-refreshes expired ID tokens transparently
Logout	Firebase Auth signOut() — clears local session and invalidates token
Server Validation	Every API request: Firebase Admin SDK verifyIdToken() — rejects expired or forged tokens

7.3 Authorization

Authorization is enforced at two independent layers — Express middleware for API routes and Firestore Security Rules for direct database access — ensuring that even a compromised API cannot grant cross-user data access.

Layer	Mechanism	Scope
API Authorization	authMiddleware.js — verifies Firebase token and attaches decoded uid to req object	All /api/* routes except /health
Data Authorization	Firestore Security Rules — match /users/{uid}/**: allow only if request.auth.uid == uid	All Firestore document reads and writes
Route Guards	ProtectedRoute.jsx — redirects unauthenticated users to /login	All React routes except /login and /register

7.4 Credential and Secret Management

© 2026 CreatorFlow AI. All rights reserved.

Golden Rule: Secrets Never Leave the Server

The OpenAI API key, Firebase Admin SDK service account credentials, and all other secrets are stored exclusively as server-side environment variables on Render. They are never included in source code, never committed to GitHub, never returned in API responses, and never referenced in frontend code. The .gitignore file includes .env on the very first commit — before any secret is written.

Secret	Storage Location	Access Pattern
OpenAI API Key	Render environment variable (OPENAI_API_KEY)	Read only in openaiService.js on server
Firebase Admin Credentials	Render environment variable (FIREBASE_SERVICE_ACCOUNT_JSON)	Read only in firebaseAdmin.js on server init
Firebase Client Config	Vercel environment variables (VITE_FIREBASE_*)	Public — contains no secrets; Firebase keys are not secret by design
.env.example	Committed to GitHub (no values)	Documents required variable names for setup

7.5 Transport Security

Transport	Security Measure
Frontend ↔ Backend	HTTPS enforced by Vercel and Render — no HTTP in production; HSTS headers configured
Backend ↔ OpenAI API	HTTPS — enforced by OpenAI SDK; TLS 1.2+ required by OpenAI
Backend ↔ Firebase	Firebase Admin SDK uses HTTPS internally for all Firestore and Auth operations
CORS Policy	Express cors() middleware — explicitly restricts allowed origins to the Vercel frontend URL only

7.6 Input Validation and Injection Prevention

Risk	Mitigation
Prompt Injection	Server-side input sanitization in promptBuilder.js — strips control characters; enforces max length
XSS (Cross-Site Scripting)	React's default JSX rendering escapes all dynamic content — no dangerouslySetInnerHTML used
Request Body Tampering	Express body-parser limits request body to 10KB; all input fields validated before prompt construction
Firestore Rule Bypass	Security rules are enforced by Firebase on every read/write — server-side SDK bypasses are scoped to Admin SDK only, not user data paths

7.7 Security Checklist

© 2026 CreatorFlow AI. All rights reserved.

Control	Status	Verified By
.gitignore with .env committed before first push	Required	Git log inspection
Firebase Auth token validated server-side on every request	Required	API integration test
Firestore security rules restrict access to document owner	Required	Firebase Rules Emulator
OpenAI API key not present in any frontend bundle	Required	Browser DevTools network inspection
CORS restricted to Vercel frontend origin	Required	Express CORS config review
HTTPS enforced in production on both Vercel and Render	Required	SSL certificate check on deploy
No plain-text passwords stored anywhere in the system	Required	Firebase Auth handles all credential storage

© 2026 CreatorFlow AI. All rights reserved.

8. Design Review and Feedback

8.1 Design Review Process

A structured self-review of this Software Design Specification was conducted following completion of the initial draft. As a solo developer, the review process used a structured checklist methodology — evaluating each section against the assignment rubric, the SRS requirements, and real-world design best practices. The review was conducted in two passes: a completeness pass (does every required section exist and cover its topic?) and a consistency pass (are design decisions consistent with each other and with the SRS?).

8.2 Review Session Notes

Area Reviewed	Finding	Severity
Section 2 — Architecture	Initial draft did not explicitly separate Firebase Auth SDK (client-side) from Firebase Admin SDK (server-side), which are distinct services used in different tiers	Medium
Section 4 — Database	Initial schema omitted the 'tone' field from GeneratedItems — needed to reconstruct generation context for saved items	Low
Section 5 — Components	useGenerate.js custom hook was added during review — the initial draft had API call logic duplicated in each page component, which violates DRY principle	Medium
Section 6 — API	Error response format was not standardized in the initial draft — different endpoints were returning inconsistent error shapes	Medium
Section 7 — Security	CORS configuration was mentioned but the specific allowed-origin scope (Vercel URL only) was not stated explicitly	Low
Overall Diagrams	Architecture diagram lacked the CORS boundary annotation between client and server tiers	Low

8.3 Revisions Made

Finding	Revision Applied
Firebase SDK split not explicit	Section 2.3 and 2.3.3 now explicitly distinguish Firebase SDK v10 (client token acquisition) from Firebase Admin SDK (server token verification), with each mapped to its correct tier
Missing 'tone' field in GeneratedItems	Section 4.3.3 GeneratedItems schema updated to include 'tone' field with enum values — preserves generation context for saved items
Duplicated API logic across components	Section 5.2 component map updated to include useGenerate.js as a shared custom hook — all four generator pages now delegate API call state management to this hook
Inconsistent error response format	Section 6.3.3 added a dedicated Error Response Format subsection with a standardized error object structure and enumerated error codes applied to all endpoints

© 2026 CreatorFlow AI. All rights reserved.

Finding	Revision Applied
CORS scope ambiguity	Section 7.5 Transport Security updated to specify 'explicitly restricts allowed origins to the Vercel frontend URL only' rather than generic CORS mention
Architecture diagram gap	Architecture diagram in Section 2.2 updated to include the HTTPS boundary between Client and Server tiers

8.4 Post-Review Design Confidence

Following the review and revision cycle, all seven user stories from the SRS remain fully traceable to design decisions in this document. The component map in Section 5.2 covers all frontend routes defined in Section 3.4. The API endpoints in Section 6.2 correspond exactly to the functional requirements FR-03 through FR-06. The database schema in Section 4.3 supports all data operations required by FR-07 and the Data Requirements in the SRS. No design gaps remain that would block Sprint 2 through Sprint 6 implementation.

© 2026 CreatorFlow AI. All rights reserved.

Document Revision History

Version	Date	Description
v1.0	June 2026	Initial Software Design Specification — all sections completed and submitted for review

© 2026 CreatorFlow AI. All rights reserved.